

ICS4U Final Project Reflection

Name:

Qianhao Han

Project Title:

MediaWise

Theme:

Media Literacy & Responsible Technology Use
Technology and App Ideas (from ICS3U list)

Sources / Build Notes:

Build Notes for the Extension:

Tools and Technologies Used:

- HTML for the popup structure
- CSS for styling the popup interface
- JavaScript for extension logic and browser interaction
- Chrome Extension Manifest V3
- Node.js and Express for the local backend server
- OpenAI API for AI text analysis and follow-up chatbot responses
- dotenv for storing the API key securely in a .env file
- ChatGPT and Claude for getting started + debugging + improving structure

Sources:

- Chrome Extensions Documentation: <https://developer.chrome.com/docs/extensions>
- Chrome Extensions Get Started: <https://developer.chrome.com/docs/extensions/get-started>
- Chrome Hello World Extension Tutorial: <https://developer.chrome.com/docs/extensions/get-started/tutorial/hello-world>
- Chrome Content Scripts Documentation: <https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>
- Chrome Message Passing Documentation: <https://developer.chrome.com/docs/extensions/develop/concepts/messaging>
- Chrome Scripting API Documentation: <https://developer.chrome.com/docs/extensions/reference/api/scripting>
- Express Documentation: <https://expressjs.com/>
- Express Hello World Example: <https://expressjs.com/en/starter/hello-world.html>
- Express Installing Guide: <https://expressjs.com/en/starter/installing.html>
- Express Routing Guide: <https://expressjs.com/en/guide/routing.html>
- OpenAI API Documentation: <https://developers.openai.com/api/docs>
- OpenAI API Quickstart: <https://developers.openai.com/api/docs/quickstart>
- OpenAI JavaScript/TypeScript SDK: <https://github.com/openai/openai-node>

I used the official Chrome Extensions documentation to learn about Manifest V3, popup files, content scripts, message passing, and loading an unpacked extension. I also used the Express documentation to understand how to create a local Node.js server, and the OpenAI API documentation to understand how to connect the backend server to the OpenAI API.

Shared GitHub Repo: (for the website)

<https://github.com/QianhaoHan/CS-Summative>

Extension Github Repo:

https://github.com/QianhaoHan/CS_extension

Link to our pitch deck: <https://canva.link/Ora3nzo27wjli0e>

(This includes screen recordings/demos of all the features)

Statistics on Media Literacy:

<https://www150.statcan.gc.ca/n1/daily-quotidien/231220/dq231220b-eng.htm#archived>

Section 1: Project and Architecture Overview

Briefly describe your final product and its purpose in your **Bobcats' Den pitch**.

In your response, explain:

- what problem your product was designed to solve
- who the target audience or market is
- how the product connects to your chosen theme
- the overall technical architecture of the program

Be sure to mention the advanced programming concepts you used, such as:

- modular design
- algorithms
- file input/output
- classes or advanced data structures
- multiple files or connected program components

Our final product MediaWise includes two deliverables: a website and a Chrome browser extension that helps users become more aware of misinformation, bias, and unreliable online content. In a time where short-form content and social media are increasingly common, many people, especially students, consume a large amount of information through news and social media posts every day, but may not always know how to tell if something is trustworthy. This is the motivation behind MediaWise. Statistics Canada found that in 2023, 43% of Canadians said it was becoming harder to distinguish between true and false information online, and 59% said they were very or extremely concerned about online misinformation. Almost 1 in 10 Canadians also said they did not know how to fact-check information.

Our goal is not just to tell users if something is true or false, but to equip them with the skills to think critically about the content they are consuming.

Our main target audience is high school students between the ages of 14-18. We chose this group because students spend the most time online and are still developing critical thinking and fact-checking skills. Our secondary audience is older adults, since they can also be affected by misinformation and may benefit from a simple tool that helps them evaluate online sources.

MediaWise connects to the provided theme of “Technology and App ideas”, with our own more specific focus on media literacy and responsible technology use. MediaWise connects to this because it uses a website and chrome extension to help users recognize misinformation, think more critically about the content they consume, and also analyze online articles in real time.

Our first deliverable, the website, uses HTML, CSS, and JavaScript. HTML organizes the content and course pages, CSS creates the visual design, and JavaScript handles the interactive parts such as quizzes and score results. On the other hand, our second deliverable, the extension, also uses HTML and CSS for the popup design, while JavaScript collects highlight text from webpages, gathers page details like the title, URL, author, and date when available, and displays the analysis back to the user. For the backend of the extension, we used [Node.js](#) and Express. [Node.js](#) allowed us to run JavaScript on the server side instead of only in the browser, and Express helped us create a backend server with routes such as /analyze and /chat. The extension sends the selected or pasted text to our backend server, and the server connects to the OpenAI API to analyze the text. We used a backend server because it keeps the API key private instead of storing it directly inside the extension. The server returns a structured response that includes risk level, category scores, bias analysis, loaded words, source credibility, verification questions, and a neutral rewrite.

We used various advanced programming concepts. For instance, our project shows a modular design because different parts of the program are separated into different files. We use algorithms for the interactive parts of our project. For example, the quiz algorithm checks the user’s answers, calculates their score, and gives feedback. In the extension, algorithms are used to organize the analysis by showing risk level, category scores, loaded words, and verification questions. We also included a fallback algorithm that scans for loaded words and weak evidence phrases if the AI analysis doesn’t work. We used file input/output and configuration through files like manifest.json, package.json, and the .env file for storing the API key. Data structures such as objects and arrays were used to store selected text, metadata, chat history, category scores, and AI responses. Overall, MediaWise consists of two main systems: one for the website, one for the Chrome extension, which includes a popup script, backend server, and AI API.

Section 2: Project Management and Leadership

Reflect on your role as the **Project Leader**.

In your response, explain:

- how you planned milestones and organized the project timeline
- how you delegated tasks to the ICS3U students
- what challenges came with organizing people, tracking progress, and combining everyone's code

- one example of how you supported, guided, or mentored your team

As the project leader, my role was to help organize our team, plan the timeline, divide up tasks, and make sure all parts of the project came together. At the beginning of the project, we brainstormed possible ideas as a group. We were originally considering an entirely different topic and spent a class or so working on it, but we realized it would be difficult to create multiple connected deliverables for it. After discussing our options, we decided on something related to media literacy because it was a topic that came up after our discussion of AI deep-fakes and was an issue we all acknowledged. It also gave us room to make different deliverables from different perspectives, including both teaching through the website and application through the extension.

One challenge we faced early on was that Jessica and I were away during the first week because of DECA ICDC. This made our timeline much tighter, so when we came back, we caught up with Juliette and created clearer deadlines as well as assigned concrete tasks. At that point, we had a little over two weeks left. Our goal was to finish most of the coding in about a week, then use the next few days to combine the separate parts, fix issues, and improve the product. After that, we focused on preparing the presentation and rehearsing.

Since MediaWise had two main deliverables, I divided the work based on both the project needs and each person's strengths/prior knowledge. Juliette worked on the diagnostic quiz because she had ideas for the questions while we were discussing. Jessica worked on the overall website layout and course pages and implemented the quiz from Juliette into the website. I chose to work on the Chrome extension because it involved tools and concepts beyond the regular Grade 11 course, such as the extension setup, backend server, and API connection, which we covered more of in the Grade 12 course. I also set up the GitHub repository and helped them debug whenever they had trouble uploading their code. One challenge with combining everyone's code was making sure the quiz, course pages, and website styling were all consistent, and I reminded them when I spotted anything that needed to be updated.

To stay organized, I created a Google Chat where we checked in frequently, especially as we got closer to the presentation date. Through the chat, they could ask me questions whenever they had trouble or had a new idea they wanted to implement. Jessica had some questions about how to organize the course pages and structure the module content, so I helped her brainstorm the topics, plan the layout and implement parts of it in the website. Since we lost some time at the start, we spent more time outside of class working together. Over the long weekend, we met twice online to rehearse, review our speaking parts, and make final edits.

Section 3: Technical Contributions and Application

Describe the advanced technical work that **you** were responsible for.

You should discuss:

- specific features or systems you built
- modular programming principles you used
- any complex data types, structures, or algorithms you developed
- how you helped keep the code organized, maintainable, and well-documented

You may also refer to:

- file structure
- reusable functions or modules
- debugging and testing
- commenting and documentation practices

In terms of the technical contributions, I was mainly responsible for building the Google Chrome extension and backend server. This included setting up the extension file structure, sending user-selected text to a server, and using the OpenAI API to generate media literacy analysis.

I organized the extension into separate files and folders. The `manifest.json` file defines the extension name, permissions, popup page, and content script. It also gives the extension permission to access the active tab and connect to the local backend server hosted on my laptop at `localhost:3000`. The popup interface is built in `popup.html`, where users can analyze selected text, paste text manually, and ask follow-up questions through the chatbot section. The styling is in a separate CSS file `style.css`, which helps keep the layout, cards, buttons, risk colors, metadata box, and score grid organized and easier to update. I also tried to make the extension user-friendly by keeping it simple, with clear buttons for analyzing selected or pasted text, separate sections for the AI analysis and chatbot, color-coded risk level, and error messages when the backend is not running properly.

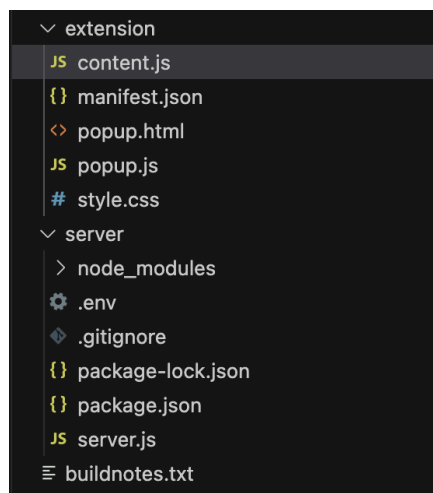
An important feature I built was the system for collecting text from a webpage. In the [content.js](#) file, the `saveSelectedText()` function stores the user's highlighted text. The function `getArticleMetadata()` collects useful page details such as the title, URL, site name, author, publication date, and description when they are available. The content script listens for messages from the popup and sends back both the selected text and metadata.

Most of the extension logic is in [popup.js](#). The `getSelectedTextFromPage()` function communicates with the content script to retrieve the highlighted text. The `analyzeText()` function then sends the text and metadata to the backend using a fetch request to the `/analyze` route. After the server responds, `renderAnalysis()` displays the result from the OpenAI API in the popup, including the risk level, category score, summary, bias analysis, loaded words, evidence quality, source credibility, verification questions, and a neutral rewrite. I also used functions like `getRiskClass()` and `getScoreClass()` to automatically apply different styling based on the risk level or score.

For the backend server, I used [Node.js](#) and Express in [server.js](#). [Node.js](#) allowed me to run JavaScript outside the browser, and Express created server routes like `/analyze` and `/chat`. The server uses `dotenv` to load the API key from a `.env` file, which keeps the key private instead of placing it directly inside the extension. The backend receives text from the extension, sends it to the OpenAI API with a built-in media literacy prompt, and returns a structured response. The dependencies and start script are also organized in `package.json`, which makes the server easier to run with `npm start`.

I also used more complex data structures and algorithms. In [server.js](#), I created a structured JSON schema called `mediaWiseAnalysisSchema` so the AI response would consistently include fields related to analyzing bias, such as `riskLevel`, `categoryScores`, `loadedWords`, and `verificationQuestions`. I also created a fallback algorithm in `getRuleBasedFallback()`, which scans the text for loaded words and weak evidence phrases if the AI request fails, then calculates a basic risk level. In [popup.js](#), I used objects and

arrays to store the current text, metadata, chat history, category scores, and rendered analysis sections.



To keep the code organized and maintainable, I separate the project into an extension folder and a server folder, as shown in my file structure. I also broke down the code into reusable functions rather than putting everything in one large code block, and made sure to add comments throughout my code. This makes debugging easier because I could test one part at a time. For example, when I first tested the extension, it was able to open and collect text, but it wasn't receiving a response from the OpenAI API. Because my code was separated clearly, I knew the issue was most likely in the backend/API request rather than the content script or popup layout. I tested each part separately, checked whether the server was running properly, looked at the /analyze route in [server.js](#) and used ChatGPT to help understand the error messages and debug the API connection.

Overall, this made the extension easier to fix and helped me turn it into a functional tool for users.

Although my main task was the Chrome extension, I also helped with parts of the website. Specifically, I helped write and set up the structure for the beginner.html and intermediate.html course pages, including the module layout and some of the course content. Jessica later modified and expanded these pages as part of her section, so I didn't explain them in this section.

Section 4: Ethics, Environment, and Emerging Technology

Reflect on the broader impact of your project and software development choices.

In your response, discuss:

- any ethical issues connected to your product or idea
- how environmental sustainability influenced your design choices or team workflow
- any links between your project and emerging technologies or computer science research

MediaWise connects to several ethical issues because it uses AI to analyze online content. One important concern is that users might rely too heavily on the tool and assume the AI's answer is always correct. To avoid this, the product is designed to guide users through the process of how to critically evaluate a source. It explains possible bias, emotional language, weak evidence, source credibility issues, and gives follow-up questions for you to consider. This encourages users to think more critically and check information themselves rather than blindly trusting the AI outputs. Another ethical concern would be AI misclassification. For instance, the tool shouldn't claim that something is definitely AI-generated or fake based only on the writing style, so the prompt tells the AI to use careful wording and focus on evidence instead.

Environmental sustainability also influenced our workflow and design choices. As a team, we mostly

shared files digitally through GitHub, Google Docs, or Canva, and communicated through Google Chat, which reduced the need for printing planning documents. In terms of the product design, we chose to keep the extension focused on analyzing selected or pasted text instead of entire webpages, which makes each request smaller and less resource-intensive. When I first started testing the extension, I also displayed how many tokens each OpenAI request used. I found that most requests ranged from 600-900 tokens, depending on how much text was selected, which was less than I expected. This helped me think more carefully about keeping requests efficient. Because AI tools can use a lot of computing power, we tried to design our extension as a tool that users would use intentionally when they need help evaluating content, instead of constantly scanning every webpage in the background.

Our project also connects strongly to emerging technologies and computer science research. MediaWise consists of both an education website with an AI-supported Chrome extension, so users can first learn media literacy skills and then apply them in real time while reading online. The extension connects to emerging AI technologies and current research areas such as natural language processing, misinformation detection, and AI-generated media. The project also connects to broader discussions about responsible AI, because it uses AI to support users in asking better questions about online information, rather than replacing human judgment altogether.

Section 5: Final Takeaways

Reflect on your learning from the full project experience.

In your response, explain:

- what this project taught you about large-scale software development
- what this project taught you about leadership and teamwork
- what you are most proud of achieving this semester

This project taught me that large-scale software development requires a lot more planning and organization than smaller, individual assignments. Because our project had multiple connected parts, including the website, quiz, course pages, and Chrome extension, I had to think more carefully about how each part would connect and how to assign tasks to each member. I learned the importance of separating code into clear files, and this process showed me that debugging is much easier when the project structure is more organized.

As a project leader, I learned a lot about leadership and teamwork through this process. I had to make sure everyone was on track to meet our deadlines and learned that delegation isn't just assigning, but I also had to make sure to check in, answer questions, help troubleshoot, and make sure that everyone's work fits together in the final product. Active communication through Google Chat and sharing files on GitHub was one of the most important parts of making this project successful, as well as being flexible and helping each other when we may be stuck in one part of the project. I am proud of how our team worked together despite the time constraints we faced. We had less class time than expected, but everyone still took responsibility for completing their parts and stayed committed to delivering a successful final product and engaging presentation!

There are many things I have been proud of throughout this course. Before this year, I didn't have much

experience with web development or using APIs, so projects like this one felt quite new to me despite my background and experience in computer science. Through our previous units, I was able to work with HTML, CSS, JavaScript, and I even explored combining a Chrome extension, backend server, and using APIs into one functioning product. I am very proud of my work with this extension because it pushed me to experiment with something new and apply the skills I learned to build a product that can actually solve a real-world issue. Overall, this project helped me become more confident with web development, API integration.